

Therefore, a great challenge of **SSL** for the future **AI** systems is (1) to represent uncertainty in the prediction and (2) to allow for multiple predictions being equally probable for a single input, i.e., *multimodal* predictions.

The approaches that have been developed so far have limitations. In particular, probabilistic models are intractable in continuous domains, meaning we do not know how to properly represent normalized continuous distributions in high dimensions. For example, we can predict the next word in the sentence because we can produce a probability distribution over all words in a dictionary and pick the most probable one. It used to be computationally challenging, but that is how modern large language models write text. However, if we want to predict the next frame in the video or the next one hundred words in the sentence, building a distribution over this enormous set of all possibilities stops being feasible. In such high-dimensional and multi-modal real-world setups, we may need to *give up the idea of representing probability distributions over predictions*.

The proposed solution is, therefore, to replace probabilistic models with energy-based models (**EBMs**) and combine them with latent variables for handling uncertainty and task-dependent information redundancy as with the hierarchical structure for complex planning within the architecture called hierarchical joint embedding predictive architecture (**H-JEPA**).

2.5 Structure of the manuscript

The following sections aim to describe every element of this fundamental building block of proposed future **AI** systems. In section 3, we introduce energy-based models (**EBMs**) and latent variable **EBMs** and compare them with the probabilistic models. In section 4, we describe how to train **EBMs** with contrastive and regularized methods, and we explain the disadvantages of the former. Section 5 presents examples of **EBMs** of historical and practical relevance, i.e., Hopfield networks, Boltzmann machines, and masking and denoising autoencoders (**AEs**). Finally, section 6 introduces the building block of the proposed **AI** of the future, that is joint embedding predictive architectures (**JEPAs**), describes their training and extension to hierarchical joint embedding predictive architectures (**H-JEPAs**) which should be able to provide multi-level and multi-timescale predictions needed for future **AI** systems.

3 Introduction to energy-based models

Probabilistic models require normalization and may therefore become intractable in the limit of high-dimensional data. However, in decision-making tasks such as driving a car, it is merely necessary that the system chooses the correct answer. The probabilities of other answers are irrelevant as long as they are smaller. Therefore, instead of predicting a single most probable event, we can let the model represent the dependency between variables (e.g., decisions y and conditions x) through the energy function, as schematically presented in Fig. 3(a). Such an energy-guided model only needs to assign the lowest energy to the correct answer and larger energies to the incorrect ones.

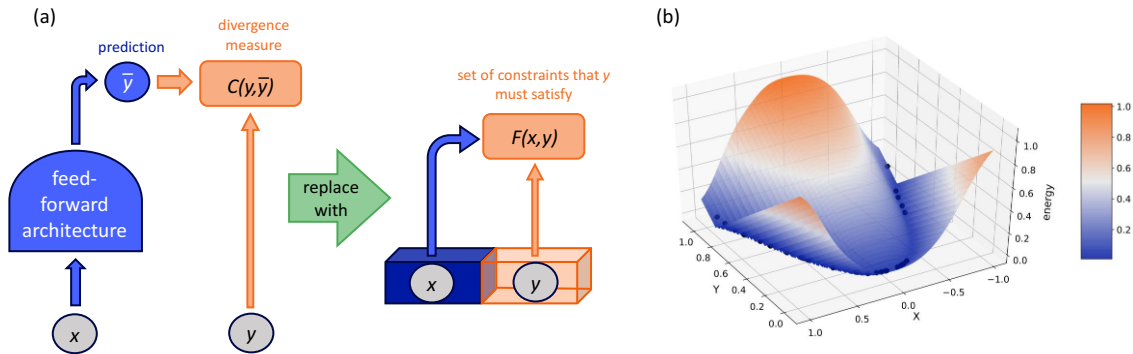


Figure 3: **Towards energy-based models (EBMs)**. (a) To achieve multimodal predictions in high dimensions, we can replace probabilistic models with **EBMs**. Then, instead of minimizing the divergence measure between the prediction and the target, we look for y 's that satisfy a set of constraints posed by x , expressed as the energy function, $F(x, y)$. A trained **EBM** should assign low energies to y 's that are a good continuation of x (in case of video or text) or that are compatible or similar (in case of images of an object taken from different angles). (b) Exemplary energy function capturing the dependence of x and y (which is $y = x^2$) from the training set, represented as blue points. The applied architecture is presented in Fig. 5(a). Note that the energy function is not unique given only the training data set!

Energy-based models (**EBMs**) capture dependencies between variables x and y by associating a scalar energy $F(x, y)$ to each configuration of the variables [37, 38]. Low energies should be assigned to compatible pairs of x and y (e.g., slow driving when snow is present) and high energies to incompatible pairs. The inference consists then in finding values of y that minimize $F_w(x, y)$: $\tilde{y} = \operatorname{argmin}_y F_w(x, y)$.

An exemplary energy function representing the quadratic dependency between x and y is presented in Fig. 3(b). The inference then involves finding the minimum energy value for a given x . An advantage of **EBMs** is that they can represent multimodal dependencies in which multiple values of y are compatible with a given x . In theory, they can also describe dependencies between data in various forms (text, visuals, etc.). Moreover, a useful property of **EBMs** is that if the y domain is continuous, the energy function F is smooth and differentiable so that we can use gradient-based inference algorithms.

An important thing to note is that the energy function discussed here is used only for inference, not learning. Training of an **EBM** (described in more detail in section 4) involves a loss function and consists in finding an energy function in which observed configurations of the variables are given lower energies than unobserved ones.

Therefore, the energy function is not the objective function to minimize within the learning! The energy is used only for inference.

3.1 Energy-based models vs. probabilistic models

In the probabilistic setting, the training consists in finding such model parameters $\mathbf{w}$ that maximize the likelihood (or minimize the negative likelihood) of observing outputs given inputs:³

$$-\log P_{\mathbf{w}}(y_1, \dots, y_n \mid \mathbf{x}_1, \dots, \mathbf{x}_n) = -\log \prod_i P_{\mathbf{w}}(y_i \mid \mathbf{x}_i) = -\sum_i \log P_{\mathbf{w}}(y_i \mid \mathbf{x}_i), \quad (1)$$

where the first equality comes from the assumption that data points are independent, and the second transformation is done because summation is easier computationally than multiplication. With probabilistic models, the training is confined to loss functions generated from the negative log-likelihood like the cross entropy. When we use other losses, we lose even this weak connection to the probabilistic setting. **EBMs**, which do not have an immediate probabilistic interpretation, give much more flexibility in the choice of the training setup and, in particular, the objective function used for learning. They also give more flexibility in the choice of the similarity/scoring function.

While it may be surprising that we give up on a probabilistic setting, note that making decisions can be viewed as choosing the option with the highest score instead of the most probable one. A well-studied example is playing chess, where deciding on the next move by looking at all possibilities and their consequences is intractable. Instead, one can explore a part of the tree of possibilities [2], e.g., with Monte-Carlo tree search or dynamical programming to find the shortest path in some graph giving, in the end, the minimal energy (or minimal energy for you and maximal for your opponent). Therefore, there is no need to use a probabilistic framework, as you just need scores for each situation (word/move, etc.), which in particular do not need to be normalized. In fact, it is hurtful to normalize those scores as we enter the label bias problem [39], which happens when the transition probabilities of leaving a given state are normalized for only that state instead of globally.

While the advantages of **EBMs** over probabilistic models are obvious in high dimensions, we lose something when giving up on the probabilistic setting. Firstly, it is challenging to understand uncertainty in an energy-based setting. Secondly, we lose outputs that can be compared between models. In particular, energies are uncalibrated (i.e., measured in arbitrary units), so combining two separately trained **EBMs** is not straightforward. To avoid this drawback, we need to design an architecture that does not require transferring outputs between its components.

However, if needed, we can make a connection between **EBMs** and probabilistic models by considering energies as the unnormalized negative log probabilities. Therefore, we can turn the **EBM** into the probabilistic model as probabilistic models are a special (normalized) case of **EBMs**. The most common method for turning a collection of arbitrary energies into a collection of numbers between 0 and 1 whose sum (or integral) is one is through the *Gibbs-Boltzmann distribution*:

$$P(y \mid x) = \frac{\exp[-\beta F(x, y)]}{\int dy' \exp[-\beta F(x, y')]}, \quad (2)$$

where the denominator is called the *partition function*, which encodes how the probabilities are partitioned among the different possibilities based on their individual energies $F(x, y)$ (therefore, plays a role of a normalization constant), and β plays a role of the inverse temperature. Computing a partition function is very challenging, so probabilistic models in continuous domains in high dimensions are intractable.

³What is also worrying, if we take a probabilistic model and train it on data by maximizing likelihood, the only model maximizing it is the one that assigns an infinite probability to every data point, and zero exactly everywhere else. It is a terrible model, and we cannot use it for inference! Of course, the Bayesian setting that assumes some prior and maximizes posterior avoids this problem by regularization.

Prediction problem	Examples of latent variables
Face recognition	the gender of the person, the orientation of their face, the color of their eyes/hair
Object recognition or scene parsing	the pose parameters of the object (location, orientation, scale), lighting conditions, segmentation of the image into component objects, assignment of labels to objects
Speech tagging	sentence segmentation into syntactic units, the parse tree
Speech recognition	sentence segmentation into phonemes or phones
Handwriting recognition	the segmentation of the line into characters

Table 1: Examples of prediction problems and potentially relevant latent variables

3.2 Latent variable energy-based models

We can expand the possibilities of **EBMs** by using an additional energy function that depends on a set of hidden variables z whose correct value is never (or rarely) given to us. Those hidden variables are often called *latent variables*, and they aim at capturing the information in y that is not readily available in x . Examples of such latent variables may be gender, pose, or hair color in the face detection task. In the case of self-driving cars, latent variables can parametrize the possible behaviors of other drivers. Thus, they give us a way of handling real-world uncertainty. More examples of latent variables are in Tab. 1. Finally, latent variables are useful in so-called *structured prediction problems*.

In a structured prediction problem, we assume that the data has some unknown structure that the learner must resolve to make an accurate prediction.

For example, in object detection, input data is structured as an image composed of multiple objects in a scene, and the goal is to recover both the structure of the image in the form of a *segmentation map* and multiple predictions for the contents of each substructure in the image. Another example of a structured prediction task is speech recognition (predicting the text transcript from an audio recording, e.g., to provide video captioning), where the missing structure is segmenting the audio into individual words. Handwriting recognition is also a structured prediction problem as to predict the text transcript from handwritten words, the algorithm needs to detect the segmentation of the handwritten word into individual letters. In the examples of structured prediction tasks listed above, the segmentation maps can naturally be viewed as latent variables for the prediction problem.

When working with latent variable **EBMs**, the inference process for a given set of variables x and y involves additionally minimizing or marginalizing⁴ with respect to these unseen variables z , and the energy function becomes either $F_w(x, y) = \operatorname{argmin}_z E_w(x, y, z)$ or $F_w(x, y) = -\frac{1}{\beta} \log \left[\int dz' \exp[-\beta E_w(x, y, z')] \right]$ as presented schematically in Fig. 4(a). The inference then is simply $\check{y} = \operatorname{argmin}_y F_w(x, y)$.

An educational example of inference with latent variable **EBMs** is an **EBM** parametrized by $w = \{r_1, r_2\}$ that learned an ellipse in the data manifold as presented in Fig. 4(b) and whose latent variable was used to encode the angle: $E_{r_1, r_2}(y, z) = (y_1 - r_1 \sin z)^2 + (y_2 - r_2 \cos z)^2$. The resulting energy function computes a squared distance between a point y to the learned manifold by minimizing the $F_{r_1, r_2}(y) = \operatorname{argmin}_z E_{r_1, r_2}(y, z)$ with respect to z which amounts to finding the distance between the data point y and its closest point of the ellipse at angle

⁴Marginalization leads to a formula known from statistical physics, where F is the free energy of the ensemble of systems with energies E , see Eq. (A5) in Appendix A. However, it becomes expensive in high dimensions and is usually replaced with a cheaper minimization instead.

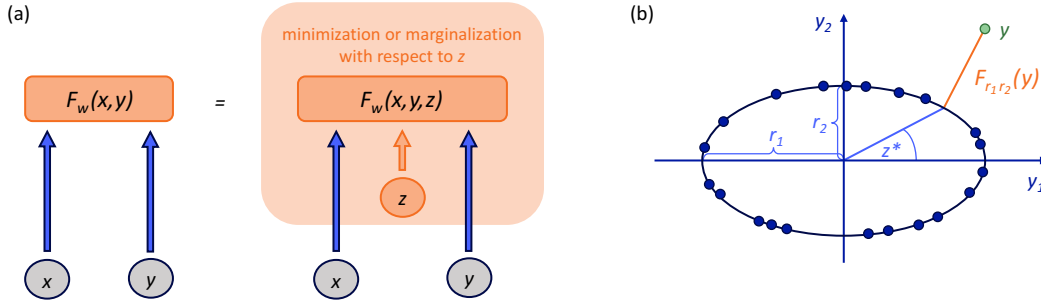


Figure 4: **Latent variable energy-based models (EBMs)**. (a) Inference in latent variable EBMs additionally includes the minimization (or marginalization) with respect to the latent variable. (b) An example of the latent variable EBM in the problem of finding the distance of a green point y from an ellipse learned from the training points depicted as blue dots. The latent variable here encoded the angle at which the closest to y point on the ellipse lies.

ǰ. We can use the same example by visualizing the difference between minimization and marginalization. If the inference is made by $F_{r_1, r_2}(y) = -\frac{1}{\beta} \log \left[\int dz' \exp \left[-\beta E_{r_1, r_2}(y, z') \right] \right]$, it is analogous to computing contributions to the energy of y from all points of the ellipse (so for z from 0 to 2π) with contributions from the closest points being the largest.

An important feature of latent variable models is that while they are composed of deterministic functions, they are non-deterministic as the latent variable they depend on is determined outside the model.

The latent variable can be drawn out of a particular probability distribution or chosen from a set of possibilities, and such sampling produces either a distribution over predictions (making the model probabilistic) or just a set of predictions (making the model only non-deterministic), respectively. Therefore, latent variables enable multiple predictions instead of just one. They can play a role of a switch or modulator, taking into account various situational factors. Ideally, the latent variable represents independent explanatory factors of prediction variation. The tricky part is that the information capacity of the latent variable must be minimized. Otherwise, the training may put all the information needed for the prediction into them.

4 Training an energy-based model

So far, we have discussed how to use EBMs, in particular, latent variable EBMs, for inference. In the case of basic EBMs, the inference consists in finding values of y that minimize the energy function $F(x, y)$: $\check{y} = \operatorname{argmin}_y F(x, y)$. When we deal with latent variable EBMs, we minimize the energy function also with respect to the latent variable z : $\check{y} = \operatorname{argmin}_{y, z} E(x, y, z)$. In this section, we describe how to train EBMs.

The main principle of training an EBM is to sculpt the energy landscape such that data points that are probable or compatible have low energy, while unlikely data points have high energy. There are two main approaches: *contrastive methods* and *regularized and architectural methods*.

The choice of the training technique depends on the choice of the EBM architecture. Let

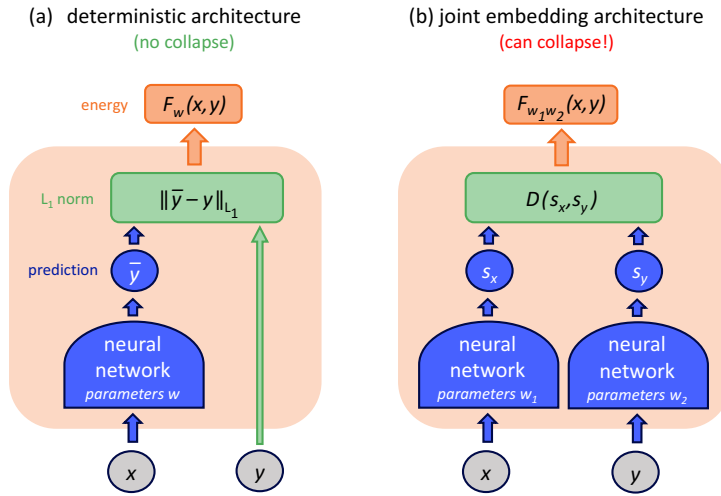


Figure 5: **Energy-based models (EBMs) can collapse.** (a) Standard deterministic architecture for prediction or regression, where energy function $F_w(x, y)$ is the distance between the NN prediction for x and the y itself, is immune to collapse. (b) An example of an EBM that can collapse.

us compare two EBM architectures in Fig. 5. In the first case, the energy function is simply the distance between the data point y and the output of an encoder (e.g., an NN) of the data point x . Such an architecture can be thought of as a regression model⁵ and trained by simply minimizing the energy of training samples. However, for other architectures, such training may lead to the *collapse* of the energy function, i.e., given an x the energy landscape could become “flat”, giving essentially the same energy to all values of y . Consider for example a joint-embedding architecture in Fig. 5(b). Such an architecture encodes the inputs x and y into $\text{Enc}_x(x)$, $\text{Enc}_y(y)$, respectively. The goal is to find such Enc_x , Enc_y so their representations of x and y are close. If we train our model only to minimize the distance between the outputs of encoders, then the two encoders are likely to ignore inputs entirely and just produce identical constant outputs.

There are more examples of EBMs that can collapse. The latent variable generative EBM architecture in Fig. 4(b), if trained to simply minimize the distance between y and latent variable-dependent prediction on x , can ignore x entirely and find such a latent variable that causes the prediction to be identical with y . Another example is autoencoder (AE) that encodes y , the decodes y into $\tilde{y}$, and if trained to minimize the distance between y and $\tilde{y}$, it can collapse because it can learn the identity function and have zero energy for all the inputs. Thus, the AE will reconstruct perfectly inputs on which it has not been trained, which we may want to avoid.

All in all, every model that can be multimodal, i.e., have multiple predictions for a single input, is susceptible to collapse.

To prevent the energy collapse, i.e., flat energy landscape, we need to constrain our energy surface so that data points have low energies but also that points outside the regions of high data density have higher energies.

⁵Such an architecture is immune to collapse but has no way of expanding predictions to be multimodal or non-deterministic.

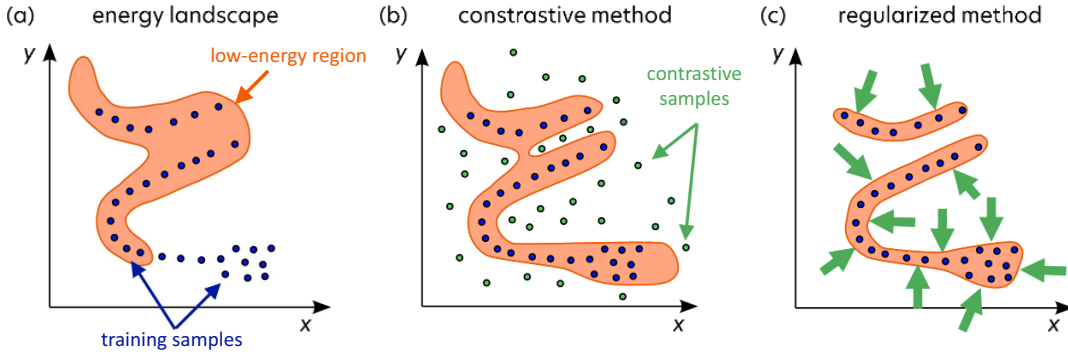


Figure 6: **Training of an energy-based model (EBM)**. (a) Proper training minimizes training samples’ energy while preventing energy collapse. (b) Contrastive methods minimize the energy of training samples while increasing the energy of samples outside the training set. (c) Regularized methods limit the volume of space that the low-energy region can take. Adapted from Ref. [6].

4.1 Contrastive methods

To prevent energy collapse, we can make use of contrastive methods.

Contrastive methods aim to push down the energy of training samples and to pull up the energy of suitably-generated contrastive samples that by definition are out of the training set. They work for any architecture but scale badly with data dimension.

An example of the contrastive loss is the following pairwise hinge loss:

$$L_{\text{hinge}}(x, y, \hat{y}, \mathbf{w}) = [F_{\mathbf{w}}(x, y) - F_{\mathbf{w}}(x, \hat{y}) + m(y, \hat{y})]^+. \quad (3)$$

x, y are training data points whose energy we aim to lower and are represented as blue dots in panel (b) of Fig. 6. $\hat{y}$ is a contrastive point, represented as green dots in panel (b) of Fig. 6, whose energy we need to increase. When minimizing L_{hinge} , we ensure that the energy of training samples is smaller than the energy of a training sample and contrastive sample, at least by the margin m that depends on the distance between y and $\hat{y}$. Suitable contrastive loss functions need to ensure a non-zero margin to avoid energy collapse. The contrastive loss functions can be calculated pairwise for specific data sets as the hinge loss in Eq. (3), but lately, a growing interest is given to group losses, which depend on a group of points instead of pairs of points, like InfoNCE [40, 41].

4.1.1 How to generate contrastive points

The heart of contrastive methods lies in how to generate contrastive points. For example, if the space of possible contrastive points is discrete or tractable, the generation of $\hat{y}$ can be exhaustive. Otherwise, they can be generated by sampling with Monte Carlo methods or their approximations [42]. Moreover, we can look for the “most offending” predictions of the model, i.e., points to which the model incorrectly assigns low energy, and choose them as contrastive points. Another approach was developed along with Siamese NNs [43], which is an example of joint-embedding architecture. There, the training set consists of both compatible and incompatible pairs, and the latter plays the role of contrastive samples. The compatible pairs can be, e.g., different views on the same object, and the incompatible data would be views on different objects. Incompatible pairs can also be generated by distorting or corrupting the original data